UTS

PEMBELAJARAN MESIN

NAMA : Sheila Fatiha Az Zahra

NIM : 202431040

KELAS : H

DOSEN : Rizqia Cahyaningtyas. ST., M.Kom

NO.PC : 07

ASISTEN : 1. Muhammad Ichwan Fauzi

2. Benedict Samuel Ardy

INSTITUT TEKNOLOGI PLN

TEKNIK INFORMATIKA

2026/2027

DAFTAR ISI

BAB I 3
PENDAHULUAN 3
1.1 Rumusan Masalah 3
1.2 Tujuan Masalah 3
1.3 Manfaat Masalah 3
BAB II 4
LANDASAN TEORI 4
2.1 Klasifikasi Pembelajaran Mesin 4
2.2 Klasifikasi pada Dataset Bank Costumers 4
2.3 Explanatory Data Analysis (EDA) 4
2.4 Preprocessing Data 4
2.5 Algoritma Naive Bayes 5
2.6 Algoritma K-Nearest Neighbors (KKN) 5
2.7 Evaluasi Model Klasifikasi 6
2.8 Ketidakseimbangan Kelas dan Pemilihan Fitur 6
BAB III 7
HASIL 7
3.1. Lingkungan dan Dataset 7
3.2. Penjelasan Setiap Cell Kode 7
BAB IV 23
PENUTUP 23
DAFTAR PUSTAKA 24

BAB I
PENDAHULUAN

1.1 Rumusan Masalah

1. Bagaimana memahami karakteristik dan struktur dataset 04_bank_customers.csv melalui proses Exploratory Data Analysis (EDA)?
2. Bagaimana melakukan preprocessing terhadap data numerik dan kategorikal agar dapat digunakan dalam proses klasifikasi?
3. Bagaimana menerapkan algoritma Gaussian Naive Bayes dan K-Nearest Neighbors (KNN) untuk mengklasifikasikan nasabah berdasarkan kemungkinan berlangganan deposito?
4. Bagaimana memperoleh dan mengevaluasi tingkat akurasi kedua algoritma menggunakan data training dan testing?
5. Bagaimana membandingkan performa Gaussian Naive Bayes dan KNN berdasarkan nilai Accuracy, Precision, Recall, F1-Score, serta Confusion Matrix?

1.2 Tujuan Masalah

1. Memahami struktur, karakteristik, dan atribut pada dataset 04_bank_customers.csv.
2. Melakukan eksplorasi data (EDA) untuk mengetahui distribusi data, karakteristik nasabah, serta hubungan antarvariabel.
3. Melakukan preprocessing data melalui pemeriksaan missing value, duplikasi, pemisahan fitur numerik dan kategorikal, encoding, serta standardisasi data.
4. Membangun dan melatih model klasifikasi menggunakan algoritma Gaussian Naive Bayes dan KNN.
5. Menguji kemampuan model dalam memprediksi status berlangganan deposito berdasarkan data testing.
6. Mengukur performa model menggunakan Accuracy, Precision, Recall, F1-Score, dan Confusion Matrix.
7. Membandingkan hasil klasifikasi kedua algoritma untuk mengetahui kelebihan dan kekurangan masing-masing metode dalam memprediksi nasabah yang berlangganan deposito.

1.3 Manfaat Masalah

1. Memahami konsep dasar Machine Learning dan penerapannya dalam proses klasifikasi data.
2. Meningkatkan kemampuan dalam melakukan pengolahan data menggunakan Python.
3. Memahami tahapan EDA, preprocessing, training, testing, dan evaluasi model.
4. Menambah pemahaman mengenai cara kerja algoritma Gaussian Naive Bayes dan KNN serta perbedaan karakteristik kedua algoritma tersebut.

BAB II
LANDASAN TEORI

2.1 Klasifikasi Pembelajaran Mesin

Pembelajaran mesin (Machine Learning) merupakan pendekatan komputasi yang memungkinkan sistem mempelajari pola data untuk menghasilkan prediksi atau keputusan tanpa seluruh aturan harus dituliskan secara manual. Dalam pembelajaran terawasi (supervised learning), model dilatih menggunakan data yang telah memiliki label target. Model mempelajari hubungan antara atribut masukan (fitur) dan label keluaran, kemudian digunakan untuk memprediksi label pada data baru. Hou et al. (2022) menerapkan beberapa algoritma supervised learning untuk memprediksi keputusan nasabah dalam berlangganan deposito berjangka dan membandingkan kinerja model yang dihasilkan.

2.2 Klasifikasi pada Dataset Bank Costumers

Klasifikasi adalah proses menempatkan suatu observasi ke dalam kelas yang telah ditentukan berdasarkan pola pada data pelatihan. Pada dataset 04_bank_customers.csv, permasalahan klasifikasi bersifat biner karena variabel target y mempunyai dua kategori, yaitu yes dan no. Kategori yes menunjukkan nasabah berlangganan deposito berjangka, sedangkan no menunjukkan nasabah tidak berlangganan. Informasi nasabah meliputi karakteristik demografis, pekerjaan, status perkawinan, pendidikan, kondisi kredit, informasi kontak, serta riwayat kampanye pemasaran.

Pemanfaatan klasifikasi pada pemasaran perbankan bertujuan membantu mengidentifikasi kelompok nasabah yang berpotensi menerima penawaran deposito. Olajide dan Andrew (2023) menjelaskan bahwa pemodelan machine learning dapat digunakan untuk memperkirakan patronase layanan deposito dan membantu bank mengenali calon nasabah. Hou et al. (2022) juga mengkaji prediksi langganan deposito menggunakan beberapa model supervised learning, termasuk Naive Bayes.

2.3 Explanatory Data Analysis (EDA)

Exploratory Data Analysis (EDA) merupakan tahap eksplorasi untuk memahami struktur, karakteristik, distribusi, dan kemungkinan masalah pada dataset sebelum pemodelan. EDA dilakukan melalui pemeriksaan tipe data, statistik deskriptif, distribusi target, distribusi fitur, serta hubungan antarvariabel. Visualisasi seperti histogram, countplot, dan heatmap membantu menemukan pola yang tidak selalu terlihat dari tabel data.

Pada dataset ini, EDA digunakan untuk melihat distribusi status langganan deposito, sebaran umur nasabah, perbandingan status langganan menurut pekerjaan, serta korelasi antarfitur numerik. Korelasi Pearson menggambarkan hubungan linear antarvariabel numerik dan tidak dapat digunakan untuk menyimpulkan hubungan sebab-akibat.

2.4 Preprocessing Data

Preprocessing merupakan tahap menyiapkan data agar sesuai dengan kebutuhan algoritma pembelajaran mesin. Tahap ini dapat meliputi pemeriksaan nilai kosong,

penghapusan duplikasi, pemilihan fitur, transformasi variabel kategorikal, penskalaan fitur numerik, serta pembagian data. Pada dataset yang dianalisis terdapat 45.211 baris dan 17 kolom. Pemeriksaan menunjukkan tidak terdapat nilai kosong bertipe NaN dan tidak terdapat baris duplikat. Nilai kategori 'unknown' tetap diperlakukan sebagai kategori yang tercatat, bukan dianggap sebagai NaN.

Fitur kategorikal perlu diubah menjadi representasi numerik agar dapat diproses model. One-Hot Encoding membentuk kolom indikator untuk setiap kategori sehingga kategori tidak diberi urutan numerik buatan. Fitur numerik distandardisasi menggunakan StandardScaler. Standardisasi mengubah fitur berdasarkan rata-rata dan simpangan baku data training, sehingga skala fitur lebih sebanding. Hal ini penting khususnya bagi KNN yang menggunakan jarak antarobservasi.

Pembagian data dilakukan menjadi 80% data training dan 20% data testing. Stratified splitting digunakan untuk mempertahankan proporsi kelas target pada kedua bagian. Transformasi dipasang di dalam pipeline agar parameter encoder dan scaler dipelajari dari data training saja. Dengan demikian, informasi dari data testing tidak digunakan saat fitting preprocessing.

2.5 Algoritma Naive Bayes

Naive Bayes merupakan algoritma klasifikasi probabilistik yang menggunakan Teorema Bayes untuk menghitung peluang suatu observasi termasuk ke dalam kelas tertentu berdasarkan fitur yang diketahui. Algoritma ini disebut 'naive' karena mengasumsikan fitur-fitur saling independen secara kondisional ketika kelas diketahui. Kelas prediksi dipilih berdasarkan probabilitas posterior terbesar. Secara umum, P(C|X) sebanding dengan P(X|C) dikalikan P(C), dengan P(C) sebagai probabilitas awal kelas dan P(X|C) sebagai probabilitas fitur pada kelas tersebut.

Implementasi yang digunakan adalah Gaussian Naive Bayes. Model ini mengasumsikan bahwa nilai setiap fitur numerik pada masing-masing kelas mengikuti distribusi Gaussian. Naive Bayes relatif sederhana dan efisien untuk klasifikasi, tetapi asumsi independensi fitur dapat tidak sepenuhnya sesuai dengan hubungan nyata antaratribut. Hou et al. (2022) memasukkan Naive Bayes dalam perbandingan model prediksi langganan deposito bank.

2.6 Algoritma K-Nearest Neighbors (KKN)

K-Nearest Neighbors merupakan algoritma klasifikasi berbasis kedekatan atau instance-based learning. Untuk mengklasifikasikan satu data baru, KNN menghitung jarak data tersebut terhadap data training, memilih sebanyak k tetangga terdekat, lalu menentukan kelas berdasarkan suara mayoritas tetangga. Pada eksperimen ini digunakan k=5 dan bobot uniform, sehingga setiap tetangga memberikan bobot suara yang sama.

Jarak Euclidean yang umum digunakan pada fitur numerik dihitung dengan akar jumlah kuadrat selisih setiap dimensi. Karena fitur dengan rentang nilai besar dapat mendominasi jarak, standardisasi fitur numerik menjadi penting. Pemilihan nilai k memengaruhi kompleksitas model: k yang terlalu kecil dapat sensitif terhadap noise, sedangkan k yang

terlalu besar dapat mengaburkan pola lokal. Bansal et al. (2022) membahas KNN sebagai salah satu algoritma yang dibandingkan dalam analisis klasifikasi machine learning.

2.7 Evaluasi Model Klasifikasi

Evaluasi dilakukan menggunakan data testing yang tidak digunakan saat pelatihan. Confusion matrix merangkum hasil prediksi menjadi True Positive (TP), True Negative (TN), False Positive (FP), dan False Negative (FN). Accuracy adalah proporsi seluruh prediksi yang benar. Precision mengukur proporsi prediksi positif yang benar, sedangkan recall mengukur proporsi seluruh data positif aktual yang berhasil ditemukan model. F1-score merupakan rata-rata harmonik precision dan recall.

Dalam algoritma ini kelas positif adalah yes. Oleh karena itu, precision, recall, dan F1-score yang dilaporkan berfokus pada kemampuan model menemukan nasabah yang benar-benar berlangganan deposito. Metrik ini penting karena jumlah kelas no jauh lebih besar daripada kelas yes. Watono et al. (2024) meneliti klasifikasi penerimaan deposito pada dataset yang tidak seimbang dan menunjukkan bahwa ketidakseimbangan kelas dapat memengaruhi performa klasifikasi; penelitian tersebut menguji teknik undersampling dan ensemble untuk mengatasinya.

2.8 Ketidakseimbangan Kelas dan Pemilihan Fitur

Ketidakseimbangan kelas terjadi ketika jumlah observasi pada satu kelas jauh lebih banyak dibandingkan kelas lainnya. Pada data ini kelas no mencapai 88,30%, sedangkan kelas yes hanya 11,70%. Akibatnya, model dapat memperoleh accuracy yang relatif tinggi dengan lebih sering memprediksi kelas mayoritas, meskipun kemampuan menemukan kelas minoritas masih rendah. Karena itu, accuracy perlu dibaca bersama precision, recall, F1-score, dan confusion matrix.

Fitur duration tidak disertakan dalam pemodelan. Durasi panggilan baru diketahui setelah percakapan telepon berlangsung. Jika tujuan penggunaan model adalah mengidentifikasi calon nasabah sebelum panggilan dilakukan, fitur tersebut berpotensi menimbulkan data leakage karena memuat informasi yang belum tersedia pada saat prediksi seharusnya dibuat. Keputusan ini merupakan pertimbangan rancangan eksperimen, bukan berarti fitur tersebut tidak berguna untuk seluruh tujuan analisis.

BAB III

HASIL

3.1. Lingkungan dan Dataset

Praktikum menggunakan Python pada Jupyter Notebook/Google Colab dengan pustaka Pandas, NumPy, Matplotlib, Seaborn, dan Scikit-learn. Dataset 04_bank_customers.csv dibaca dengan pemisah titik koma. Ukuran awal dataset adalah 45.211 baris dan 17 kolom. Variabel target adalah y dengan kelas no dan yes.

3.2. Penjelasan Setiap Cell Kode

- **Cell 2 – Import Library**

  Mengimpor pustaka untuk pengolahan data, visualisasi, pembagian data, preprocessing, pemodelan Gaussian Naive Bayes dan KNN, serta evaluasi. pandas dan numpy mengolah data; matplotlib dan seaborn membuat grafik; scikit-learn menyediakan fungsi machine learning. RANDOM_STATE=42 digunakan agar pembagian data dapat diulang. matplotlib.pyplot diimpor dua kali, tetapi pengulangan ini tidak memengaruhi hasil.

```
# 202431040

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report, ConfusionMatrixDisplay
)

import warnings
warnings.filterwarnings('ignore')

RANDOM_STATE = 42
```

- **Cell 4 – Load Dataset**

  Membaca file 04_bank_customers.csv menggunakan pd.read_csv() dengan sep=';' karena file menggunakan pemisah titik koma. Data disimpan dalam variabel data dan ditampilkan. Dataset berisi 45.211 baris dan 17 kolom.

Load Dataset

```
# 202431040

data = pd.read_csv('04_bank_customers.csv', sep=';')
data
```

	age	job	marital	education	default	balance	housing	loan	contact	day	mon
0	58	management	married	tertiary	no	2143	yes	no	unknown	5	m
1	44	technician	single	secondary	no	29	yes	no	unknown	5	m
2	33	entrepreneur	married	secondary	no	2	yes	yes	unknown	5	m
3	47	blue-collar	married	unknown	no	1506	yes	no	unknown	5	m
4	33	unknown	single	unknown	no	1	no	no	unknown	5	m
...	...	...	...	...	...	...	...	...	...	...	
45206	51	technician	married	tertiary	no	825	no	no	cellular	17	n
45207	71	retired	divorced	primary	no	1729	no	no	cellular	17	n
45208	72	retired	married	secondary	no	5715	no	no	cellular	17	n
45209	57	blue-collar	married	secondary	no	668	no	no	telephone	17	n
45210	37	entrepreneur	married	secondary	no	2971	no	no	cellular	17	n

45211 rows × 17 columns

- **Cell 6 – Menampilkan Lima Baris Pertama**
  data.head() menampilkan lima baris pertama untuk melihat contoh isi dataset, nama atribut, dan bentuk nilai pada setiap kolom.

```
# 202431040
data.head()
```

	age	job	marital	education	default	balance	housing	loan	contact	day	month	du
0	58	management	married	tertiary	no	2143	yes	no	unknown	5	may	
1	44	technician	single	secondary	no	29	yes	no	unknown	5	may	
2	33	entrepreneur	married	secondary	no	2	yes	yes	unknown	5	may	
3	47	blue-collar	married	unknown	no	1506	yes	no	unknown	5	may	
4	33	unknown	single	unknown	no	1	no	no	unknown	5	may	

- **Cell 7 – Informasi Struktur Dataset**
  data.info() menampilkan jumlah baris, nama kolom, jumlah nilai non-null, dan tipe data. Dataset memiliki 17 kolom, terdiri dari 7 kolom numerik dan 10 kolom kategorikal/object.

```
# 202431040
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 45211 entries, 0 to 45210
Data columns (total 17 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   age        45211 non-null  int64
 1   job        45211 non-null  object
 2   marital    45211 non-null  object
 3   education  45211 non-null  object
 4   default    45211 non-null  object
 5   balance    45211 non-null  int64
 6   housing    45211 non-null  object
 7   loan       45211 non-null  object
 8   contact    45211 non-null  object
 9   day        45211 non-null  int64
 10  month      45211 non-null  object
 11  duration   45211 non-null  int64
 12  campaign   45211 non-null  int64
 13  pdays      45211 non-null  int64
 14  previous   45211 non-null  int64
 15  poutcome   45211 non-null  object
 16  y          45211 non-null  object
dtypes: int64(7), object(10)
memory usage: 5.9+ MB
```

- **Cell 8 – Statistik Deskriptif**
  data.describe() menampilkan statistik ringkas kolom numerik, seperti jumlah data, rata-rata, standar deviasi, nilai minimum, kuartil, dan maksimum. Hasilnya membantu memahami rentang dan sebaran atribut numerik.

```
# 202431040
data.describe()
```

	age	balance	day	duration	campaign	pdays	p
count	45211.000000	45211.000000	45211.000000	45211.000000	45211.000000	45211.000000	45211
mean	40.936210	1362.272058	15.806419	258.163080	2.763841	40.197828	0
std	10.618762	3044.765829	8.322476	257.527812	3.098021	100.128746	2
min	18.000000	-8019.000000	1.000000	0.000000	1.000000	-1.000000	0
25%	33.000000	72.000000	8.000000	103.000000	1.000000	-1.000000	0
50%	39.000000	448.000000	16.000000	180.000000	2.000000	-1.000000	0
75%	48.000000	1428.000000	21.000000	319.000000	3.000000	-1.000000	0
max	95.000000	102127.000000	31.000000	4918.000000	63.000000	871.000000	275

- **Cell 10 – Pemeriksaan Missing Value dan Duplikasi**
  Cell bermaksud menghitung nilai kosong dengan isnull().sum() dan baris duplikat dengan duplicated().sum(). Namun, kode menggunakan df, sedangkan DataFrame dibuat dengan nama data. Jika df belum didefinisikan, cell akan mengalami NameError. Perbaikan: ganti df menjadi data. Pada dataset yang diperiksa, missing value berjumlah 0 dan duplikasi berjumlah 0.

```
# 202431040

print("Jumlah Missing Value:")
print(df.isnull().sum())

print("\nJumlah Data Duplikat:")
print(df.duplicated().sum())
```

```
Jumlah Missing Value:
age          0
job          0
marital      0
education    0
default      0
balance      0
housing      0
loan         0
contact      0
day          0
month        0
duration     0
campaign     0
pdays        0
previous     0
poutcome     0
y            0
dtype: int64

Jumlah Data Duplikat:
0
```

- **Cell 11 – Identifikasi Fitur Numerik**
  select_dtypes(include=np.number) memilih kolom numerik dan menyimpannya dalam numerical_features. Cell menampilkan jumlah, nama kolom, dan lima baris pertama fitur numerik. Dataset memiliki 7 kolom numerik sebelum target dan duration dikeluarkan dari fitur model.

```
# 202431040
numerical_features = data.select_dtypes(include=np.number).columns.tolist()
print("Jumlah fitur numerik:", len(numerical_features))
print("Daftar fitur numerik:", numerical_features)
display(data[numerical_features].head())
```

```
Jumlah fitur numerik: 7
Daftar fitur numerik: ['age', 'balance', 'day', 'duration', 'campaign', 'pday
s', 'previous']
```

	age	balance	day	duration	campaign	pdays	previous
0	58	2143	5	261	1	-1	0
1	44	29	5	151	1	-1	0
2	33	2	5	76	1	-1	0
3	47	1506	5	92	1	-1	0
4	33	1	5	198	1	-1	0

- **Cell 12 – Identifikasi Fitur Kategorikal**
  select_dtypes(exclude=np.number) memilih kolom bukan numerik dan menyimpannya dalam categorical_features. Cell menampilkan jumlah, nama kolom, dan lima baris pertama fitur kategorikal. Hasil identifikasi awal mencakup 10 kolom, termasuk target y.

```
# 202431040
categorical_features = data.select_dtypes(exclude=np.number).columns.tolist()
print("Jumlah fitur kategorikal:", len(categorical_features))
print("Daftar fitur kategorikal:", categorical_features)
display(data[categorical_features].head())
```

```
Jumlah fitur kategorikal: 10
Daftar fitur kategorikal: ['job', 'marital', 'education', 'default', 'housing',
'loan', 'contact', 'month', 'poutcome', 'y']
```

	job	marital	education	default	housing	loan	contact	month	poutcome	y
0	management	married	tertiary	no	yes	no	unknown	may	unknown	no
1	technician	single	secondary	no	yes	no	unknown	may	unknown	no
2	entrepreneur	married	secondary	no	yes	yes	unknown	may	unknown	no
3	blue-collar	married	unknown	no	yes	no	unknown	may	unknown	no
4	unknown	single	unknown	no	no	no	unknown	may	unknown	no

- **Cell 13 – Melihat Nilai Unik Fitur Kategorikal**
  Perulangan menampilkan nilai unik dari setiap kolom dalam categorical_features menggunakan unique(). Hasilnya membantu mengenali kategori pada atribut seperti job, marital, education, contact, poutcome, dan y.

```
# 202431040
for col in categorical_features:
    print(f"\nKolom: {col}")
    print(data[col].unique())
```

```
Kolom: job
['management' 'technician' 'entrepreneur' 'blue-collar' 'unknown'
 'retired' 'admin.' 'services' 'self-employed' 'unemployed' 'housemaid'
 'student']

Kolom: marital
['married' 'single' 'divorced']

Kolom: education
['tertiary' 'secondary' 'unknown' 'primary']

Kolom: default
['no' 'yes']

Kolom: housing
['yes' 'no']

Kolom: loan
['no' 'yes']

Kolom: contact
['unknown' 'cellular' 'telephone']

Kolom: month
['may' 'jun' 'jul' 'aug' 'oct' 'nov' 'dec' 'jan' 'feb' 'mar' 'apr' 'sep']

Kolom: poutcome
['unknown' 'failure' 'other' 'success']

Kolom: y
['no' 'yes']
```

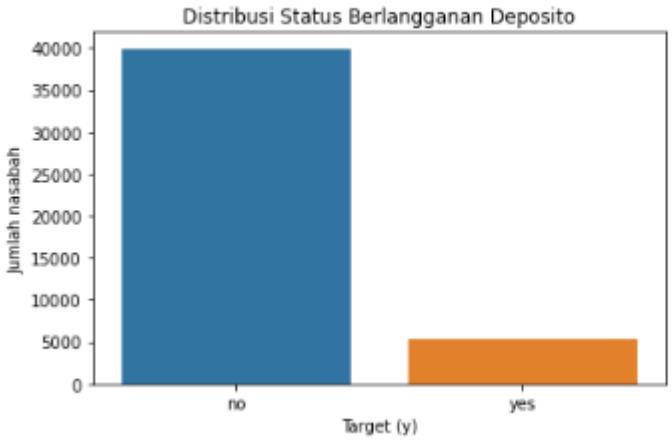
- **Cell 14 – EDA Distribusi Target**
  value_counts() menghitung jumlah kelas no dan yes, sedangkan normalize=True menghitung persentasenya. Hasil dataset adalah no sebanyak 39.922 (88,30%) dan yes sebanyak 5.289 (11,70%). Countplot memperlihatkan bahwa kelas no jauh lebih dominan, sehingga dataset mengalami ketidakseimbangan kelas (class imbalance).

EDA Distrubusi Target

```
# 202431040
print(data['y'].value_counts())
print((data['y'].value_counts(normalize=True) * 100).round(2))

sns.countplot(data=data, x='y')
plt.title('Distribusi Status Berlangganan Deposito')
plt.xlabel('Target (y)')
plt.ylabel('Jumlah nasabah')
plt.tight_layout()
plt.show()
```

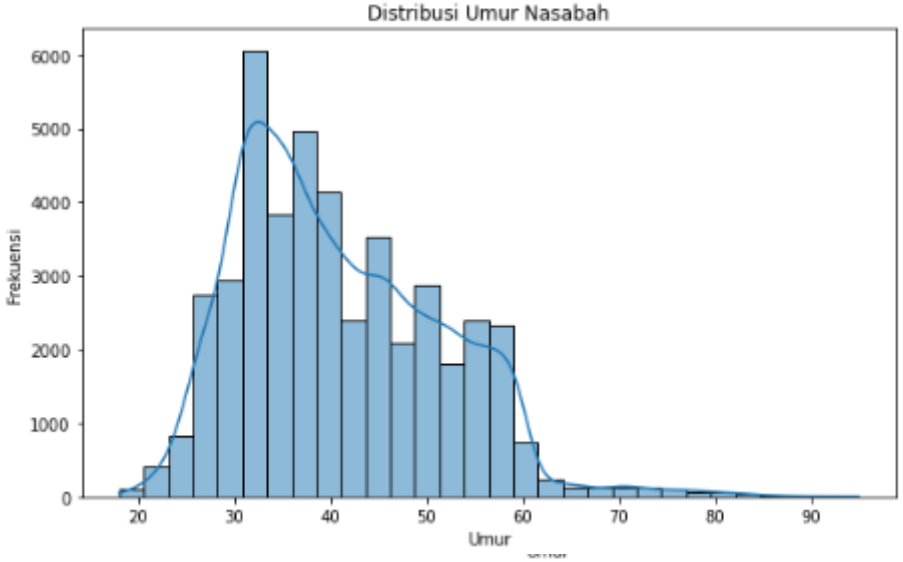
```
no     39922
yes     5289
Name: y, dtype: int64
no     88.3
yes    11.7
Name: y, dtype: float64
```



- **Cell 15 – EDA Distribusi Umur**
  Histogram dengan 30 interval dan kurva KDE menggambarkan distribusi umur nasabah. describe() menampilkan statistik umur. Pada dataset, rata-rata umur sekitar 40,94 tahun, median 39 tahun, dan rentang umur 18–95 tahun.

EDA Distribusi umur

```
# 202431040
plt.figure(figsize=(8, 5))
sns.histplot(data=data, x='age', bins=30, kde=True)
plt.title('Distribusi Umur Nasabah')
plt.xlabel('Umur')
plt.ylabel('Frekuensi')
plt.tight_layout()
plt.show()
display(data['age'].describe())
```




```
count    45211.000000
mean        40.936210
std         10.618762
min         18.000000
25%         33.000000
50%         39.000000
75%         48.000000
max         95.000000
Name: age, dtype: float64
```

- **Cell 16 – Menentukan Fitur dan Target Naive Bayes**
  X dibuat dengan menghapus kolom y dan duration. Kolom y menjadi target dan dikonversi menjadi no=0 serta yes=1. Kolom duration dikeluarkan karena durasi panggilan baru diketahui setelah panggilan berlangsung; jika prediksi dilakukan sebelum menghubungi nasabah, kolom ini berpotensi menyebabkan data leakage.

```
# 202431040
X = data.drop(columns=['y', 'duration'])
y = data['y'].map({'no': 0, 'yes': 1})

print("Fitur prediktor:", X.columns.tolist())
print("\nDistribusi target:")
print(y.value_counts())
```

```
Fitur prediktor: ['age', 'job', 'marital', 'education', 'default', 'balance',
'housing', 'loan', 'contact', 'day', 'month', 'campaign', 'pdays', 'previous',
'poutcome']

Distribusi target:
0    39922
1     5289
Name: y, dtype: int64
```

- **Cell 17 – Membagi Data Training dan Testing**
  train_test_split membagi data menjadi 80% training dan 20% testing. random_state=42 membuat pembagian konsisten, sedangkan stratify=y menjaga proporsi kelas pada kedua bagian. Hasil pembagian adalah 36.168 data training dan 9.043 data testing.

```
# 202431040
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=RANDOM_STATE,
    stratify=y
)

print("X_train:", X_train.shape)
print("X_test :", X_test.shape)
print("y_train:", y_train.shape)
print("y_test :", y_test.shape)
```

```
X_train: (36168, 15)
X_test : (9043, 15)
y_train: (36168,)
y_test : (9043,)
```

- **Cell 18 – Preprocessing Data**
  OneHotEncoder mengubah kategori menjadi kolom indikator dan handle_unknown='ignore' menangani kategori yang tidak ditemukan saat training. StandardScaler menstandarkan fitur numerik. ColumnTransformer menerapkan transformasi sesuai tipe data. Ekspresi numerical_features[:-0] if False else ... selalu mengambil bagian setelah else karena kondisi False; sehingga kolom numerik tetap dipilih dari X. Preprocessor ini digunakan di dalam pipeline model.

```
# 202431040
try:
    encoder_nb = OneHotEncoder(handle_unknown='ignore', sparse_output=False)
except TypeError:
    encoder_nb = OneHotEncoder(handle_unknown='ignore', sparse=False)

preprocessor_nb = ColumnTransformer(
    transformers=[
        ('numerik', StandardScaler(), numerical_features[:-0] if False else X.se
        ('kategori', encoder_nb, X.select_dtypes(exclude=np.number).columns.toli
    ]
)

print("Preprocessing siap digunakan.")
```

```
Preprocessing siap digunakan.
```

- **Cell 19 – Training Model Gaussian Naive Bayes**
  Pipeline menggabungkan preprocessing dengan GaussianNB. fit(X_train, y_train) melatih model menggunakan data training. GaussianNB menggunakan asumsi distribusi Gaussian untuk fitur numerik. Setelah pelatihan selesai, model siap digunakan untuk memprediksi data testing.

```
# 202431040
model_nb = Pipeline(steps=[
    ('preprocessing', preprocessor_nb),
    ('classifier', GaussianNB())
])

model_nb.fit(X_train, y_train)
print("Model Gaussian Naive Bayes berhasil dilatih.")
```

```
Model Gaussian Naive Bayes berhasil dilatih.
```

- **Cell 20 – Prediksi Data Testing Naive Bayes**
  model_nb.predict(X_test) menghasilkan prediksi kelas untuk data testing. Label 0 menunjukkan nasabah tidak berlangganan deposito (no), sedangkan 1 menunjukkan nasabah berlangganan (yes). Cell menampilkan 15 prediksi pertama.

```
# 202431040
y_pred_nb = model_nb.predict(X_test)
print("Lima belas hasil prediksi pertama:", y_pred_nb[:15])
```

```
Lima belas hasil prediksi pertama: [0 0 0 0 0 0 0 1 0 0 0 0 0 0 0]
```

- **Cell 21 – Evaluasi Naive Bayes**
  Cell menghitung Accuracy, Precision, Recall, dan F1-Score dengan membandingkan y_test dan y_pred_nb. Classification report menampilkan metrik untuk kelas No dan Yes. Hasil eksekusi: Accuracy 84,52%, Precision kelas Yes 37,08%, Recall kelas Yes 46,41%, dan F1-Score kelas Yes 41,23%.

```
# 202431040
print("Accuracy :", accuracy_score(y_test, y_pred_nb))
print("Precision:", precision_score(y_test, y_pred_nb, zero_division=0))
print("Recall   :", recall_score(y_test, y_pred_nb, zero_division=0))
print("F1-Score :", f1_score(y_test, y_pred_nb, zero_division=0))

print("\nClassification Report:")
print(classification_report(
    y_test, y_pred_nb,
    labels=[0, 1], target_names=['No', 'Yes'],
    zero_division=0
))
```

```
Accuracy : 0.8451841203140551
Precision: 0.37084592145015105
Recall   : 0.46408317580340264
F1-Score : 0.4122586062132662

Classification Report:
              precision    recall  f1-score   support

          No       0.93      0.90      0.91      7985
         Yes       0.37      0.46      0.41      1058

    accuracy                           0.85      9043
   macro avg       0.65      0.68      0.66      9043
weighted avg       0.86      0.85      0.85      9043
```

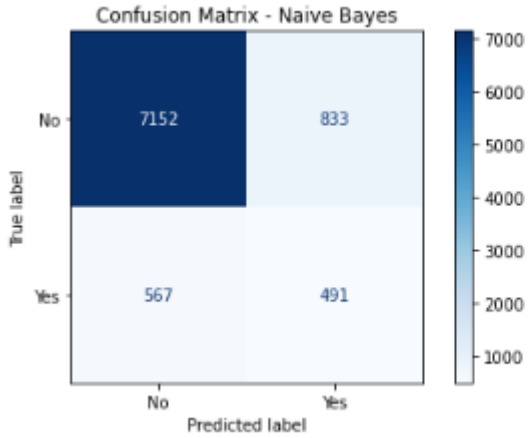
- **Cell 22 – Confusion Matrix Naive Bayes**
  Confusion matrix membandingkan label aktual dengan prediksi. Hasilnya TN=7.152, FP=833, FN=567, dan TP=491. TN adalah no yang diprediksi no; FP adalah no yang diprediksi yes; FN adalah yes yang diprediksi no; TP adalah yes yang diprediksi yes. Heatmap menampilkan nilai tersebut secara visual.

```
# 202431040
cm_nb = confusion_matrix(y_test, y_pred_nb, labels=[0, 1])
print("Confusion Matrix Naive Bayes:")
print(cm_nb)

tn, fp, fn, tp = cm_nb.ravel()
print(f"TN={tn}, FP={fp}, FN={fn}, TP={tp}")

ConfusionMatrixDisplay(
    confusion_matrix=cm_nb,
    display_labels=['No', 'Yes']
).plot(cmap='Blues', values_format='d')
plt.title('Confusion Matrix - Naive Bayes')
plt.tight_layout()
plt.show()
```

```
Confusion Matrix Naive Bayes:
[[7152  833]
 [ 567  491]]
TN=7152, FP=833, FN=567, TP=491
```



- **Cell 23 – Menampilkan Kembali Data**
  data.head() menampilkan kembali lima baris pertama dataset sebelum bagian KNN. Cell memastikan data yang digunakan pada tahap berikutnya masih berasal dari dataset yang sama.

K-Nearest Neighbors (KNN)

```
# 202431040
data.head()
```

	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	ca
0	58	4	1	2	0	2143	1	0	2	5	8	261	
1	44	9	2	1	0	29	1	0	2	5	8	151	
2	33	2	1	1	0	2	1	1	2	5	8	76	
3	47	1	1	3	0	1506	1	0	2	5	8	92	
4	33	11	2	3	0	1	0	0	2	5	8	198	

- **Cell 24 – Menyiapkan Fitur dan Target KNN**
  X_knn dibuat dengan menghapus kolom y dan duration. y_knn menjadi target dengan pemetaan no=0 dan yes=1. Fitur dan target dibuat sama seperti Naive Bayes agar perbandingan algoritma lebih sebanding.

```
# 202431040
X_knn = data.drop(columns=['y', 'duration'])
y_knn = data['y'].map({'no': 0, 'yes': 1})

print("Fitur KNN:", X_knn.columns.tolist())
print("Distribusi target:")
print(y_knn.value_counts())
```

```
Fitur KNN: ['age', 'job', 'marital', 'education', 'default', 'balance', 'housin
g', 'loan', 'contact', 'day', 'month', 'campaign', 'pdays', 'previous', 'poutco
me']
Distribusi target:
0    39922
1     5289
Name: y, dtype: int64
```

- **Cell 25 – Membagi Data Training dan Testing KNN**
  train_test_split membagi data KNN menjadi 80% training dan 20% testing, menggunakan random_state=42 serta stratify=y_knn. Variabel terpisah digunakan agar pembagian KNN tidak menimpa variabel Naive Bayes.

```
# 202431040
X_train_knn, X_test_knn, y_train_knn, y_test_knn = train_test_split(
    X_knn, y_knn,
    test_size=0.20,
    random_state=RANDOM_STATE,
    stratify=y_knn
)

print("X_train KNN:", X_train_knn.shape)
print("X_test KNN :", X_test_knn.shape)
```

```
X_train KNN: (36168, 15)
X_test KNN : (9043, 15)
```

- **Cell 26 – Preprocessing KNN: Encoding dan Standardisasi**
  OneHotEncoder mengubah fitur kategorikal menjadi numerik, sedangkan StandardScaler menstandarkan fitur numerik. Standardisasi penting untuk KNN karena algoritma menghitung jarak; fitur dengan skala besar dapat mendominasi jarak jika tidak distandarkan.

```
# 202431040
try:
    encoder_knn = OneHotEncoder(handle_unknown='ignore', sparse_output=False)
except TypeError:
    encoder_knn = OneHotEncoder(handle_unknown='ignore', sparse=False)

preprocessor_knn = ColumnTransformer(
    transformers=[
        ('numerik', StandardScaler(),
         X_knn.select_dtypes(include=np.number).columns.tolist()),
        ('kategori', encoder_knn,
         X_knn.select_dtypes(exclude=np.number).columns.tolist())
    ]
)

print("Preprocessing KNN siap digunakan.")
```

```
Preprocessing KNN siap digunakan.
```

- **Cell 27 – Training Model KNN dengan k=5**
  Pipeline menggabungkan preprocessing dan KNeighborsClassifier dengan n_neighbors=5 serta weights='uniform'. Model mempertimbangkan lima tetangga terdekat dan memberi bobot suara yang sama kepada setiap tetangga. fit() melatih model dengan data training.

Training Model KNN dengan k=5

```
# 202431040
model_knn = Pipeline(steps=[
    ('preprocessing', preprocessor_knn),
    ('classifier', KNeighborsClassifier(n_neighbors=5, weights='uniform'))
])

model_knn.fit(X_train_knn, y_train_knn)
print("Model KNN dengan k=5 berhasil dilatih.")
```

```
Model KNN dengan k=5 berhasil dilatih.
```

- **Cell 28 – Prediksi Data Testing KNN**
  model_knn.predict(X_test_knn) menghasilkan prediksi kelas untuk data testing. Hasil disimpan pada y_pred_knn dan 15 prediksi pertama ditampilkan. Nilai 0 berarti no dan 1 berarti yes.

```
# 202431040
y_pred_knn = model_knn.predict(X_test_knn)
print("Lima belas hasil prediksi pertama:", y_pred_knn[:15])
```

```
Lima belas hasil prediksi pertama: [0 0 0 0 0 0 0 1 0 0 0 0 0 0 0]
```

- **Cell 29 – Evaluasi Model KNN**
  Cell menghitung Accuracy, Precision, Recall, dan F1-Score serta menampilkan classification report. Hasil eksekusi KNN k=5: Accuracy 88,74%, Precision kelas Yes 55,29%, Recall kelas Yes 19,75%, dan F1-Score kelas Yes 29,11%. Akurasi lebih tinggi dari Naive Bayes, tetapi recall kelas Yes lebih rendah.

```
# 202431040
print("Accuracy :", accuracy_score(y_test_knn, y_pred_knn))
print("Precision:", precision_score(y_test_knn, y_pred_knn, zero_division=0))
print("Recall   :", recall_score(y_test_knn, y_pred_knn, zero_division=0))
print("F1-Score :", f1_score(y_test_knn, y_pred_knn, zero_division=0))

print("\nClassification Report:")
print(classification_report(
    y_test_knn, y_pred_knn,
    labels=[0, 1], target_names=['No', 'Yes'],
    zero_division=0
))
```

```
Accuracy : 0.8874267389140772
Precision: 0.5529100529100529
Recall   : 0.19754253308128544
F1-Score : 0.29108635097493035

Classification Report:
              precision    recall  f1-score   support

          No       0.90      0.98      0.94      7985
         Yes       0.55      0.20      0.29      1058

    accuracy                           0.89      9043
   macro avg       0.73      0.59      0.61      9043
weighted avg       0.86      0.89      0.86      9043
```

- **Cell 30 – Confusion Matrix KNN**

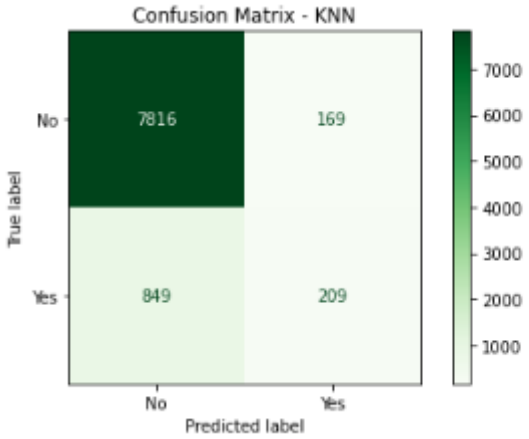
Confusion matrix KNN menghasilkan TN=7.816, FP=169, FN=849, dan TP=209. Nilai FN yang tinggi menunjukkan banyak nasabah yang sebenarnya Yes diprediksi sebagai No. Heatmap menampilkan jumlah prediksi benar dan salah.

```
# 202431040
cm_knn = confusion_matrix(y_test_knn, y_pred_knn, labels=[0, 1])
print("Confusion Matrix KNN:")
print(cm_knn)

tn, fp, fn, tp = cm_knn.ravel()
print(f"TN={tn}, FP={fp}, FN={fn}, TP={tp}")

ConfusionMatrixDisplay(
    confusion_matrix=cm_knn,
    display_labels=['No', 'Yes']
).plot(cmap='Greens', values_format='d')
plt.title('Confusion Matrix - KNN')
plt.tight_layout()
plt.show()
```

```
Confusion Matrix KNN:
[[7816  169]
 [ 849  209]]
TN=7816, FP=169, FN=849, TP=209
```



- **Cell 31 – Menguji Beberapa Nilai k pada KNN**
  Cell mentransformasikan data training dan testing menggunakan preprocessor_knn. Encoder dan scaler di-fit pada data training saja, lalu diterapkan pada data testing. KNN diuji pada k=3, 5, 7, 9, 11, 13, dan 15. Hasil accuracy berturut-turut adalah 88,19%; 88,74%; 89,07%; 89,12%; 89,20%; 89,22%; dan 89,21%. Grafik memperlihatkan pengaruh jumlah tetangga terhadap accuracy.

```
# 202431040

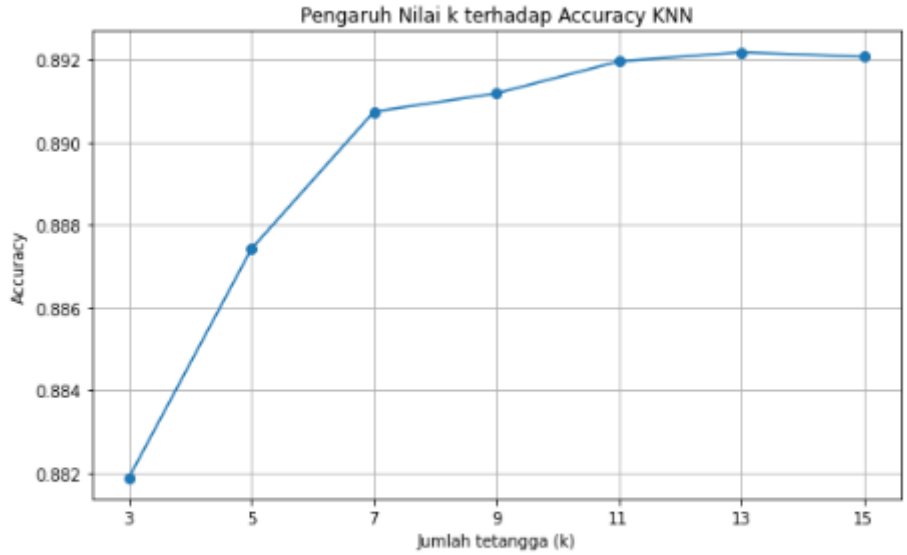
# Data ditransformasikan satu kali; scaler/encoder hanya fit pada training.
X_train_knn_transformed = preprocessor_knn.fit_transform(X_train_knn)
X_test_knn_transformed = preprocessor_knn.transform(X_test_knn)

k_values = range(3, 16, 2)
accuracies = []

for k in k_values:
    knn_temp = KNeighborsClassifier(n_neighbors=k)
    knn_temp.fit(X_train_knn_transformed, y_train_knn)
    pred_temp = knn_temp.predict(X_test_knn_transformed)
    accuracies.append(accuracy_score(y_test_knn, pred_temp))

plt.figure(figsize=(8, 5))
plt.plot(list(k_values), accuracies, marker='o')
plt.title('Pengaruh Nilai k terhadap Accuracy KNN')
plt.xlabel('Jumlah tetangga (k)')
plt.ylabel('Accuracy')
plt.xticks(list(k_values))
plt.grid(True)
plt.tight_layout()
plt.show()

print("Accuracy setiap k:", dict(zip(k_values, accuracies)))
```




```
Accuracy setiap k: {3: 0.8818976003538649, 5: 0.8874267389140772, 7: 0.89074422
20502045, 9: 0.8911865531350216, 11: 0.8919606325334513, 13: 0.892181798075859
7, 15: 0.8920712153046555}
```

- **Cell 32 – Evaluasi KNN dengan Nilai k Terbaik**
  Cell memilih k dengan accuracy tertinggi menggunakan argmax(accuracies). Hasil pengujian menunj ukkan k=13. Model dilatih kembali menggunakan data training yang telah ditransformasi, kemudian memprediksi data testing dan menampilkan classification report. Walaupun judul Markdown menyebut Confusion Matrix, kode cell ini tidak membuat confusion matrix; kode hanya menampilkan classification report.

```python
# 202431040
best_k = list(k_values)[int(np.argmax(accuracies))]
print("Nilai k dengan accuracy tertinggi pada percobaan:", best_k)

best_knn = KNeighborsClassifier(n_neighbors=best_k)
best_knn.fit(X_train_knn_transformed, y_train_knn)
best_pred = best_knn.predict(X_test_knn_transformed)

print(classification_report(
    y_test_knn, best_pred,
    labels=[0, 1], target_names=['No', 'Yes'],
    zero_division=0
))
```

```
Nilai k dengan accuracy tertinggi pada percobaan: 13
              precision    recall  f1-score   support

          No       0.90      0.99      0.94      7985
         Yes       0.64      0.18      0.28      1058

    accuracy                           0.89      9043
   macro avg       0.77      0.58      0.61      9043
weighted avg       0.87      0.89      0.86      9043
```

BAB IV

PENUTUP

Berdasarkan pengolahan dataset Bank Customers, dapat disimpulkan bahwa proses klasifikasi memerlukan EDA, preprocessing, pembagian data, pelatihan model, dan evaluasi. Dataset terdiri dari 45.211 baris dan 17 kolom, tanpa missing value NaN maupun data duplikat. Target tidak seimbang karena kelas no sebesar 88,30% dan yes sebesar 11,70%.

Dua algoritma yang diuji menghasilkan kinerja berbeda. KNN memperoleh accuracy tertinggi sebesar 88,74%, sedangkan Gaussian Naive Bayes memperoleh accuracy 84,52%. Namun, Naive Bayes menghasilkan recall kelas yes 46,41% dan F1-score 41,23%, lebih tinggi daripada KNN yang memperoleh recall 19,75% dan F1-score 29,11%. Dengan demikian, pemilihan model terbaik bergantung pada tujuan klasifikasi. Jika prioritasnya mendeteksi calon nasabah yang berpotensi berlangganan, Naive Bayes lebih sesuai pada eksperimen ini karena lebih banyak mengenali kelas yes. Hasil ini masih dapat ditingkatkan melalui pencarian parameter dan penanganan class imbalance.

Penggunaan kolom duration ditiadakan karena prediksi dirancang untuk dilakukan sebelum percakapan telepon. Keputusan ini membantu menghindari data leakage. Pengujian lanjutan tetap diperlukan sebelum model diterapkan pada data operasional bank.

DAFTAR PUSTAKA

[1] S. Hou, Z. Cai, J. Wu, H. Du, and P. Xie, “Applying Machine Learning to the Development of Prediction Models for Bank Deposit Subscription,” International Journal of Business Analytics, vol. 9, no. 1, pp. 1–14, 2022. https://doi.org/10.4018/IJBAN.288514

[2] B. O. Olajide and I. W. Andrew, “Bank Term Deposit Service Patronage Forecasting using Machine Learning,” Vallis Aurea, vol. 9, no. 2, pp. 53–64, 2023. https://doi.org/10.2507/IJVA.9.2.5.106

[3] B. Watono, E. Utami, and D. Ariatmanto, “Stacking Ensemble Learning and Instance Hardness Threshold for Bank Term Deposit Acceptance Classification on Imbalanced Dataset,” Jurnal Teknik Informatika (JUTIF), vol. 5, no. 4, pp. 453–467, 2024. https://doi.org/10.52436/1.jutif.2024.5.4.2022

[4] M. Zaki, N. Khodadadi, W. H. Lim, and S. K. Towfek, “Predictive Analytics and Machine Learning in Direct Marketing for Anticipating Bank Term Deposit Subscriptions,” American Journal of Business and Operations Research, vol. 11, no. 1, pp. 65–72, 2024. https://doi.org/10.54216/AJBOR.110110

[5] M. Bansal, A. Goyal, and A. Choudhary, “A Comparative Analysis of K-Nearest Neighbor, Genetic, Support Vector Machine, Decision Tree, and Long Short Term Memory Algorithms in Machine Learning,” Decision Analytics Journal, vol. 3, p. 100071, 2022. https://doi.org/10.1016/j.dajour.2022.100071